Craft Foods Inventory and Demand Forecasting

Project Report

Group 22

Pranavi Katikaneni
Tanaya Kondejkar

+1 (857) 313-0786
+1(949) 439-8042

katikaneni.p@northeastern.edu
kondejkar.t@northeastern.edu

Percentage of Effort Contributed by Student1: 50

Percentage of Effort Contributed by Student2: 50

Signature of Student 1: Pranavi Katikaneni

Signature of Student 2: Tanaya Kondejkar

Submission Date: 04/21/2024

Craft Foods Inventory and Demand Forecasting

I. Introduction:

Problem Statement:
A food manufacturing company "Craft Foods" faces challenges in optimizing inventory levels and accurately forecasting demand to ensure efficient production and minimize waste. Inefficient inventory management poses risks such as stockouts or surplus stock, which not only affect profitability but also customer satisfaction. Thus, the primary objective of this project is to develop and implement a comprehensive data management system tailored for inventory control and demand forecasting within a food manufacturing setting. The project aims to elevate operational efficiency, mitigate costs, and ultimately optimize the company's performance in meeting market demands.

Theory for Inventory Management and Demand Forecasting:
In the realm of inventory management and demand forecasting, the foundational principle revolves around the crucial role of accurate demand prediction. Essentially, the project recognizes that the ability to forecast demand effectively is indispensable for maintaining optimal inventory levels, which in turn directly impacts operational efficiency, cost-effectiveness, and customer satisfaction within the food manufacturing industry. To achieve this, the project aims to develop forecasting models that draw upon a multifaceted array of data sources. These models will not only rely on historical sales data, which provides valuable insights into past consumption patterns, but will also integrate analyses of prevailing market trends and consideration of external factors that may influence demand fluctuations. These external factors could include economic indicators, seasonal variations, competitor activities, and even social or cultural events. This database will serve as a repository for storing and organizing crucial information related to inventory levels, sales transactions, and forecasting data. By centralizing this data, the company can streamline operations and facilitate easier access to relevant information for decision-making purposes.

Three main tables will be required to effectively manage inventory, sales, and forecasting data. The Inventory Table will store information such as Product ID, Product Name, Current Stock, Reorder Level, Safety Stock, and Supplier Information. The Sales Table will capture details of each transaction, including Transaction ID, Product ID, Quantity Sold, Sales Date, and Customer Information. Lastly, the Forecasting Table will contain data related to forecasted demand, including Product ID, Forecasted Demand, and Forecasting Date.

By ensuring data quality through thorough cleaning and preprocessing, the company can enhance the accuracy and reliability of its inventory and sales records, leading to more informed business decisions. To safeguard sensitive information and prevent unauthorized access, the project will

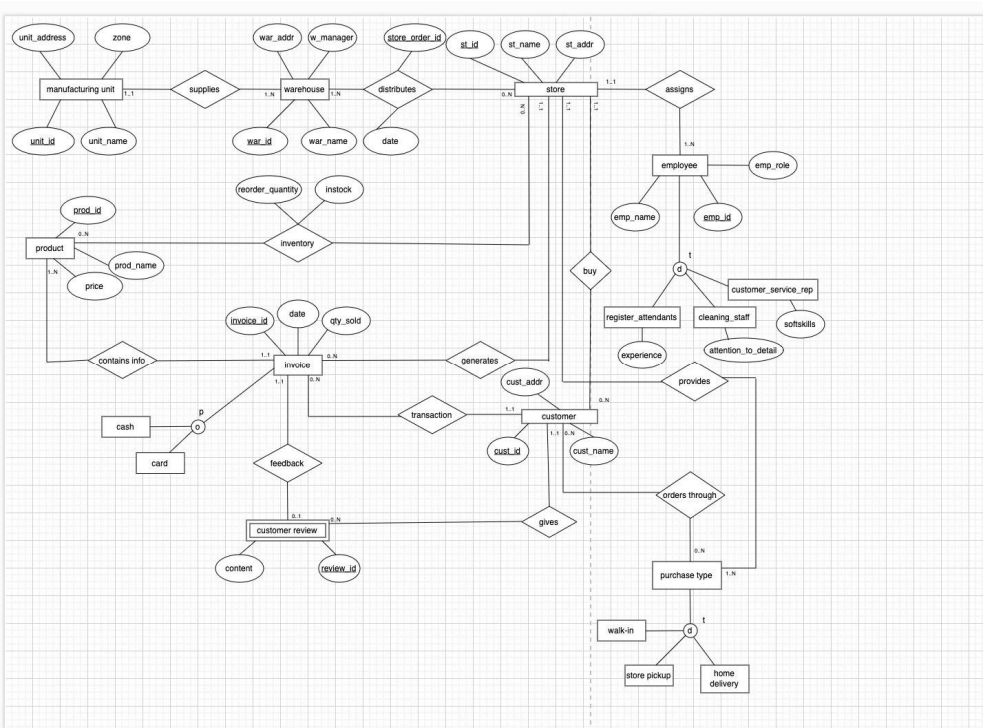
enforce proper access controls and encryption mechanisms. By implementing stringent security measures, the company can protect its data assets from potential breaches or unauthorized usage, thereby ensuring confidentiality and integrity of the information stored within the database.

Other Requirements:

1. The system must possess the capability to handle numerous products along with their associated data.
2. Provide real-time visibility into inventory levels, allowing for accurate tracking of products at each stage of the supply chain
3. The raw material originates from a single source and is then transported to the distribution centre
4. A customer may request various items from the same category, but the store either has a limited quantity or is currently out of stock. Out of stock is a Boolean value.
5. Customer feedback is recorded in understanding potential shifts in demand

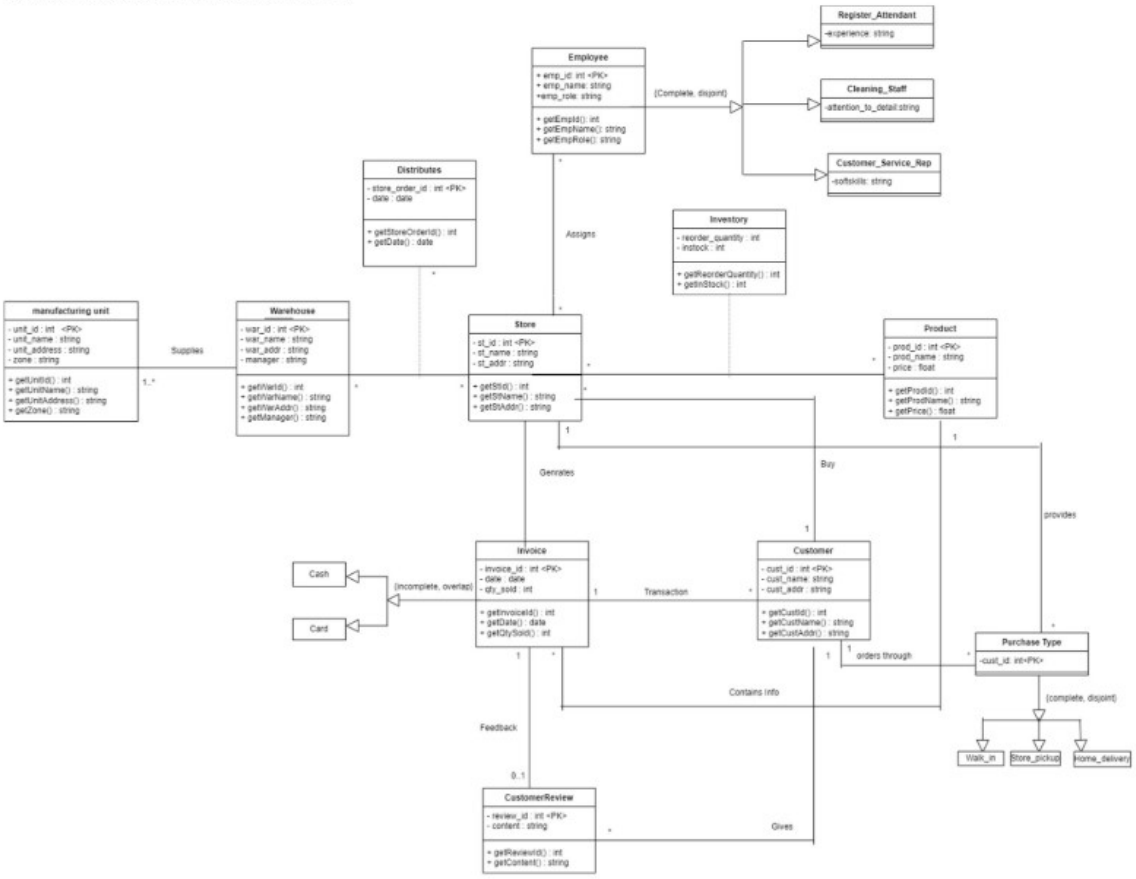
II. Conceptual Data Modelling:

1. EER Diagram:



2. UML Diagram:

Craft Foods UML DIAGRAM:



Relationships:

1. Manufacturing - Warehouse (One-to-Many):
   - Single manufacturer supplies to multiple warehouses
   - A warehouse gets its supplies from one manufacturing unit
2. Warehouse - Store (Many-to-Many):
   - Warehouses can distribute its product to multiple stores and store can get products from multiple warehouses
3. Customer - Store (One-to-Many):
   - A single customer can buy products from one store at a time
   - stores can have multiple customers at a time
4. Store - Product (Many-to-Many):
   - A store will have multiple products and a product in available in many store
5. Invoice - Customer (One-to-Many):
   - Customer can buy multiple products at a time which implies an invoice can have multiple
   - products at a time
6. Product - Invoice (One-to-Many):

- A product can be bought by a customer i.e a single product can be purchased by only one invoice

7. Customer - Customer Review (One-to-Many):
   - A customer can provide various feedbacks for multiple transactions
   - Given feedback belongs to one customer
8. Invoice - Customer Review (One-to-One):
   - each invoice corresponds to one customer review
   - a single invoice can have one or no feedback

There is a ternary relationship between store, customer, customer review and invoice

III. Mapping conceptual model to Relational Model:

Primary key is underlined and Foreign Key is in Italics.

1. Manufacturing_unit(unit_id,unit_name,unit_address,zone)
2. Warehouse(war_id,war_name,war_addr,w_manager,*unit_id*)
3. distributes(store_order_id,date,*war_id,st_id*)
4. distributes_tracking(store_order_id,date)
5. Store(st_id,st_name,st_addr)
6. Employee(*emp_id*,emp_name,emp_role,*st_id*)
7. Register_attendants(emp_id, emp_name,emp_role,experience)
8. Cleaning_staff(emp_id, emp_name,emp_role,attention_to_detail)
9. Customer_service_rep(emp_id, emp_name,emp_role,softskills)
10. Inventory(reorder_quantity,instock,*st_id,prod_id*)
11. Product(prod_id,prod_name,price)
12. Invoice(invoice_id,date,qty_sold,*prod_id,cust_id*)
13. Invoice_tracker(invoice_id,*prod_id,cust_id*)
14. Customer(cust_id,cust_name,cust_addr)
15. Purchase_type(purchase_id,*st_id,cust_id*)
16. Customer_review(review_id,content,*cust_id,invoice_id*)

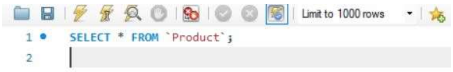
IV. Mapping of Relational Model via MySQL and NoSQL:

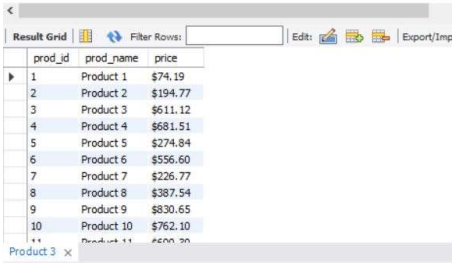
MySQL Implementation:

1.Simple Query:

Retrieve all products:

SELECT * FROM `Product`;





prod_id	prod_name	price
1	Product 1	$74.19
2	Product 2	$194.77
3	Product 3	$611.12
4	Product 4	$681.51
5	Product 5	$274.84
6	Product 6	$556.60
7	Product 7	$226.77
8	Product 8	$387.54
9	Product 9	$830.65
10	Product 10	$762.10

2.Correlated Query:

Retrieve products along with the total quantity sold for each product:

```
SELECT p.prod_name, (
 SELECT SUM(i.qty_sold)
 FROM `invoice` i
 WHERE i.prod_id = p.prod_id
) AS total_quantity_sold
FROM `Product` p;
```

```
SELECT p.prod_name, (
    SELECT SUM(i.qty_sold)
    FROM `invoice` i
    WHERE i.prod_id = p.prod_id
) AS total_quantity_sold
FROM `Product` p;
```

Result Grid | Filter Rows: | Export:

prod_name	total_quantity_sold
Product 1	139
Product 2	NULL
Product 3	NULL
Product 4	82
Product 5	145
Product 6	164
Product 7	31
Product 8	97
Product 9	NULL
Product 10	NULL
Product 11	NULL

Result 12

3.Group By Query:

Count the number of invoices for each product:

SELECT prod_id, COUNT(*) AS num_invoices
FROM `invoice`
GROUP BY prod_id;

```
SELECT prod_id, COUNT(*) AS num_invoices
FROM `invoice`
GROUP BY prod_id;
```

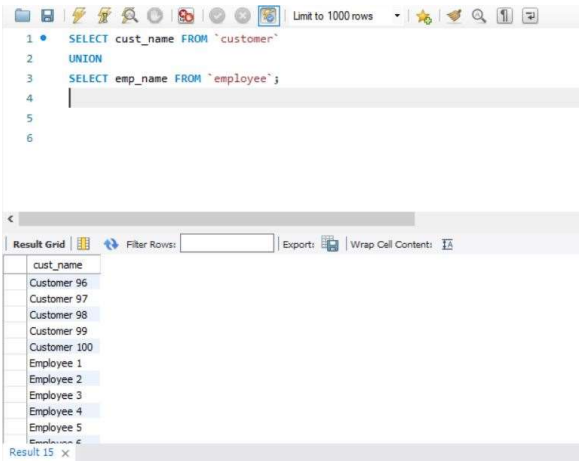
Result Grid | Filter Rows: | Export: | Wrap Cell Content:

prod_id	num_invoices
1	3
4	1
5	2
6	2
7	1
8	1
12	1
13	2
15	1
16	1

4. Union Query:

Retrieve the names of all customers and all employees in a single result set:

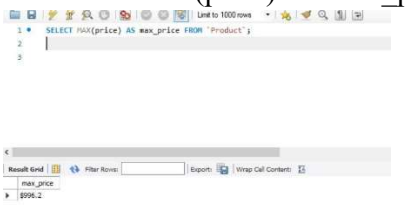
SELECT cust_name FROM `customer`
UNION
SELECT emp_name FROM `employee`;



5. Max Query:

Retrieve the maximum price among all products:

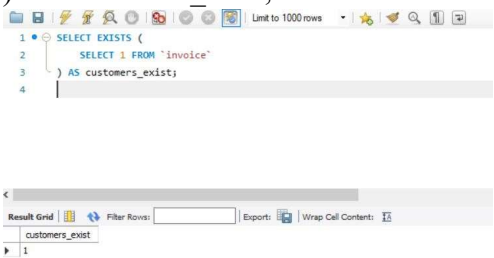
SELECT MAX(price) AS max_price FROM `Product`;



6. Exists Query:

Check if any customers have made purchases:

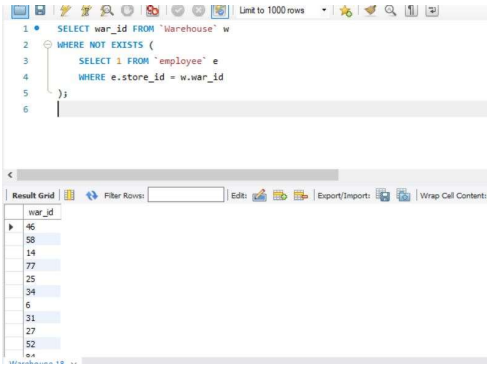
SELECT EXISTS (
 SELECT 1 FROM `invoice`
) AS customers_exist;



7. Not exists Query:

Check if there are any warehouses without managers:

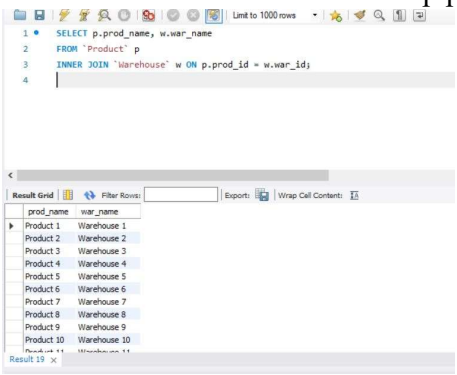
SELECT war_id FROM `Warehouse` w
WHERE NOT EXISTS (
 SELECT 1 FROM `employee` e
 WHERE e.store_id = w.war_id
);



8. Inner Join:

Retrieve products along with the warehouses where they are stored:

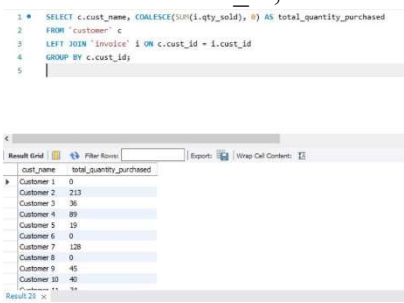
SELECT p.prod_name, w.war_name
FROM `Product` p
INNER JOIN `Warehouse` w ON p.prod_id = w.war_id;



9. Outer Join:

Retrieve all customers and the total quantity of products they have purchased, including customers who haven't made any purchases:

SELECT c.cust_name, COALESCE(SUM(i.qty_sold), 0) AS total_quantity_purchased
FROM `customer` c
LEFT JOIN `invoice` i ON c.cust_id = i.cust_id
GROUP BY c.cust_id;

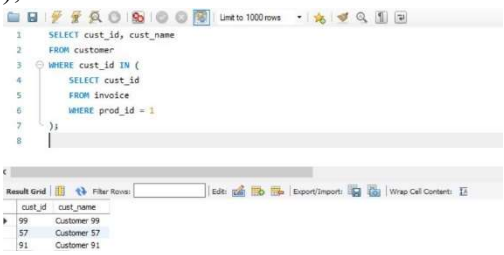


10. Nested Query:

Retrieves customer IDs and names from the "customer" table for customers who have made purchases (as indicated by their presence in the "invoice" table) of a product with ID 1.

SELECT cust_id, cust_name

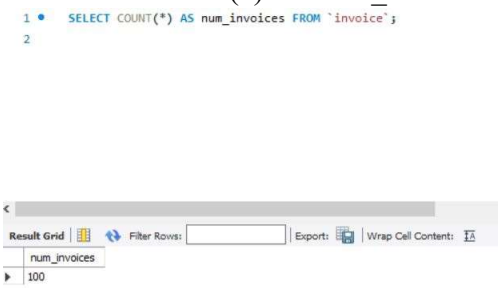
FROM customer
WHERE cust_id IN (
 SELECT cust_id
 FROM invoice
 WHERE prod_id = 1
);



11. Aggregation:

Count the number of invoices:

SELECT COUNT(*) AS num_invoices FROM `invoice`;



<u>NoSQL Implementation</u>:

1. Creation of tables:

2. Displaying All customers:

3. Products having quantity grater than 15:

4. What is the total quantity sold of all products in each invoice?

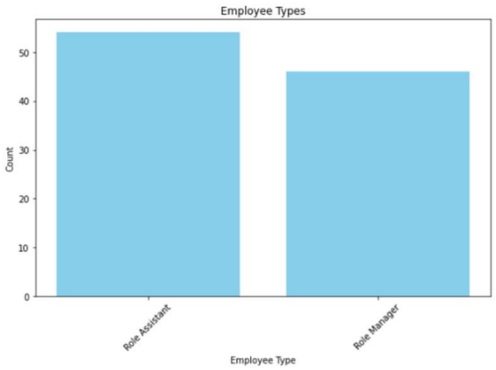
5. What are the names of all products with a price greater than $25?

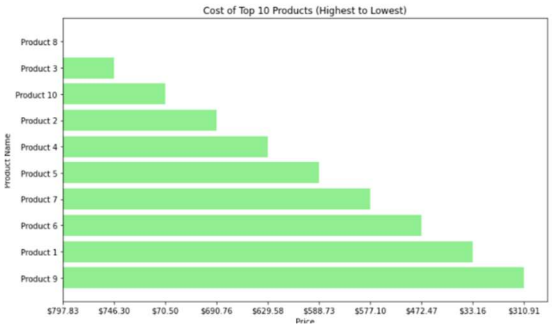
V. Application via python:

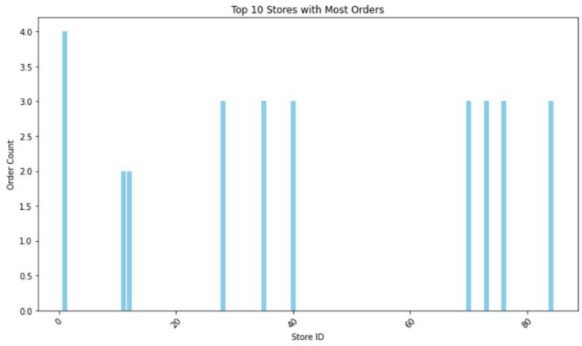
The database created for the project is accessed using Python in python IDLE. MySQL is connected to python using mysql.connector followed by cursor.execute to run and fetch all data from queries. Also, we have used matplotlib to visualise the promeient trends in data to understand it better.

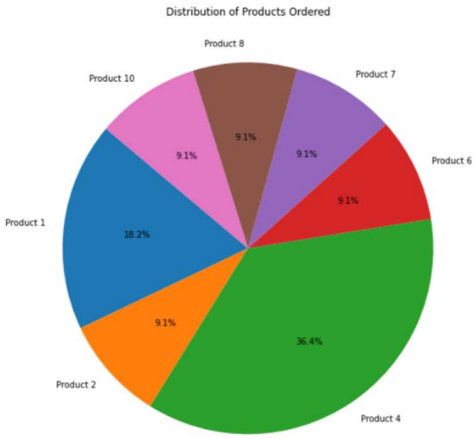
```
In [1]: pip install mysql-connector-python

        Requirement already satisfied: mysql-connector-python in c:\users\tanay\anaconda3\lib\site-packages (8.3.0)
        Note: you may need to restart the kernel to use updated packages.
```

```
In [2]: import mysql.connector

In [3]: # Establish connection
        connection = mysql.connector.connect(
            host="localhost",
            port=3309,  # If using a non-default port
            user="root",
            password="heyitsme12",
            database="dmaproject"
        )

In [4]: # Create a cursor object using the cursor() method
        cursor = connection.cursor()

In [5]: if connection.is_connected():
            db_Info = connection.get_server_info()
            print("Connected to MySQL Server version ", db_Info)
            cursor = connection.cursor()
            cursor.execute("select database();")
            record = cursor.fetchone()
            print("Your connected to database: ", record)

        Connected to MySQL Server version  8.0.34
        Your connected to database:  ('dmaproject',)
```









VII. Conclusion:

In conclusion, the successful completion of the data management system for inventory control and demand forecasting has brought significant benefits to "Craft Foods." The new system has optimized inventory levels and reduced waste through accurate demand forecasting based on various data sources. Key achievements include real-time tracking of inventory levels and products throughout the supply chain, leading to efficient production planning and decision-making. Centralized data access has streamlined operations and improved strategic choices. By incorporating customer feedback, "Craft Foods" has become more responsive to shifts in demand. The system's robust security measures protect sensitive information and ensure data integrity.